ClaimSage — The Build-It-Yourself Learning Guide

A from-scratch guide to building a production-shaped **RAG (Retrieval-Augmented Generation)** application — and, just as important, **every bug we actually hit while building and deploying this one**, with the symptom, the root cause, the fix, and the lesson.

This is written to be **revised before an interview**. If someone asks *"walk me through a RAG system you built and what broke,"* everything you need is here.

- Part 1 explains the **concepts** (what RAG is and why each piece exists).
- Part 2 is a **step-by-step build order** you could repeat on a new project.
- Part 3 is the **deployment** story (going from localhost to a public URL).
- Part 4 is the **bug journal** — the most valuable part for interviews.
- Part 5 is a **rapid-fire interview Q&A** cheat sheet.
- **Part 6 is a complete from-absolute-scratch course** for someone who has **never written code** and wants to build RAG systems and configure LLMs. It assumes *nothing*, teaches the theory in depth, and every idea comes with runnable code. **If you're a beginner, start there and come back to Parts 1–5 afterward.**

For how *this* codebase is wired, read `ARCHITECTURE.md` alongside this. For the deploy runbook, see `DEPLOY.md`.

Part 1 — The concepts

1.1 The problem RAG solves

You have large documents (a 100-page insurance policy, an Excel schedule of benefits, a folder of claim emails) and you want plain-English answers grounded in them: *"Is knee surgery covered, and what's the waiting period?"*

The naive approach — paste the whole policy into an LLM and ask — fails for three reasons:

1. **Context window limits.** The document is too big to fit in the prompt.
2. **Cost & latency.** Sending everything on every question is slow and expensive.
3. **Hallucination.** With no grounding, the model invents plausible-sounding answers.

RAG fixes all three: instead of sending the whole document, you *retrieve* only the handful of relevant passages and send those. The model reads a page, not a book.

1.2 RAG in two phases

```
INGESTION (once per document)
  file → parse to text → chunk into pieces → embed each chunk → store vectors

QUERY (every question)
  question → embed → find nearest chunks → LLM writes answer from those chunks
```

The key mental model: the LLM never reads the whole document. Retrieval does the hard part (finding the right 5 chunks); the LLM does the easy part (summarize those 5 chunks). That's why a small, free model is "good enough" here.

1.3 The two model types (don't confuse them)

Beginners conflate these. They are different models doing different jobs:

- **Embedding model** — turns text into a vector (a list of numbers, e.g. 2048 of them). Similar meanings land at nearby points in space. Used to *find* relevant chunks by measuring distance. Runs on **every chunk at ingestion** and **every question at query time**.
- **Chat / LLM model** — writes the natural-language answer. Runs **once per question**, over the retrieved chunks only.

1.4 Why a vector store (Qdrant) *and* a relational DB (SQLite)?

- **Qdrant** answers one question extremely fast: *"which stored vectors are nearest to this one?"* That's approximate-nearest-neighbour search. It is not the right tool for bookkeeping.
- **SQLite** tracks the relational bookkeeping: which sessions exist, which documents belong to them, and whether each document has finished processing (`processing` / `ready` / `failed`).

Different jobs, different databases. A common interview question is *"why not just use one?"* — because "find similar vectors" and "track upload status" are different access patterns, and forcing one store to do both makes both worse.

1.5 What makes answers trustworthy: grounding + structural citations

Two design decisions separate a toy from something you'd demo:

1. **Grounding.** The system prompt tells the LLM to answer *using only the provided context.* If the answer isn't in the retrieved chunks, it should say so rather than invent one.
2. **Structural citations.** We do **not** ask the LLM to cite itself (models lie about their sources). Instead, citations are read from the **metadata of the retrieved chunks** — filename, page label, similarity score. That metadata was attached at parse time. This is honest: the citation literally *is* the chunk that was fed to the model.

Part 2 — Build order (how to recreate this)

Build in this order. Each step is independently testable, which is the whole point — you never debug ten things at once.

Step 1 — Project skeleton

- FastAPI app, a health endpoint (`GET /api/health`), a Dockerfile, and a `docker-compose.yml` that runs Qdrant locally. Prove `/api/health` returns `{"status":"ok"}` before writing any RAG code.

Step 2 — Config from environment

- One `Settings` object (pydantic-settings) that reads `.env` : API keys, Qdrant URL, model names, chunk size. **Never hard-code secrets or hosts.** Everything that differs between your laptop and prod must be an env var. (This bites you later if you skip it — see Bug #4 and #5.)

Step 3 — The provider abstraction

- A single module (`llm_provider.py`) that is the *only* place that knows you use OpenRouter. Everything else asks it for "an LLM" or "an embedding model." When you later switch providers or add an agent layer, you edit one file.

Step 4 — Parsing (multi-format → text + metadata)

- Turn each upload into text **plus location metadata** (this metadata is what makes citations possible):
  - PDF → one unit per **page** (PyMuPDF), tagged with the page number.
  - Excel → one unit per **row** (columns flattened), so a benefits-schedule row is never split in half.
  - Email → headers + body; parse attachments recursively.
- Reject formats you can't handle *clearly* (legacy `.doc` / `.msg`) rather than producing garbage.

Step 5 — Chunking

- Split long text into ~512-token chunks with ~50-token **overlap**. Overlap stops a sentence's meaning from being cut across a boundary. Keep short units (Excel rows) whole.

Step 6 — Embedding + storage

- Embed each chunk and upsert into a Qdrant collection, tagging every vector with `claim_id` (= the session id) so you can scope retrieval later.
- Discover the embedding dimension at runtime and create the collection with it — don't hard-code 1536/2048.

Step 7 — Async ingestion

- Uploading returns **immediately** with status `processing`; the slow parse→chunk→embed work runs in a background task and flips the row to `ready` / `failed`. The client **polls** `GET /documents/{id}/status`. Doing the work inside the HTTP request would time out on a big PDF.

Step 8 — Query

- Embed the question, ask Qdrant for the `top_k` nearest chunks **filtered to this** `claim_id`, feed them to the LLM with a grounding prompt, and return the answer plus citations built from chunk metadata.

Step 9 — OCR fallback (scanned docs & photos)

- Some PDF pages are scans with **no text layer**, and users upload **photos** of documents. Add a per-page fallback: if a page yields fewer than N characters of real text, rasterize it (PyMuPDF) and run **Tesseract** OCR on the image. Crucially, OCR runs **only on pages that need it** — digital pages stay fast and accurate. Expose `?ocr_mode=auto|always|off`.

Step 10 — Frontend

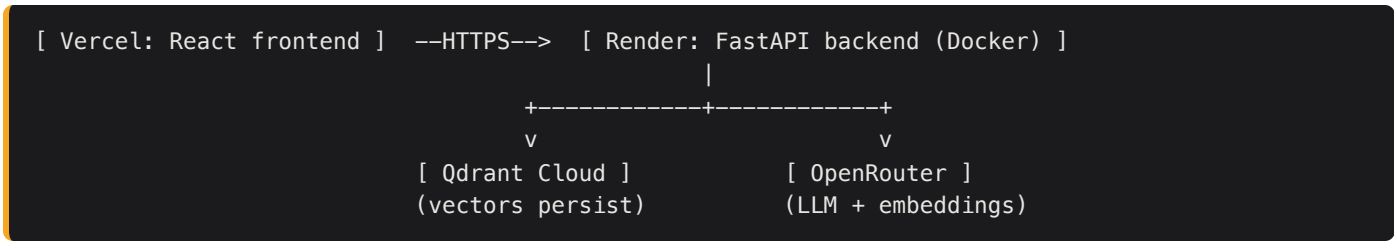
- A typed API client (base URL from an env var), a session/upload/chat UI, and status polling. Keep it dependency-light so you fully own and understand it.

Step 11 — Tests, then deploy

- Test the pipeline locally (scoped retrieval, citation correctness) *before* touching the cloud. Then Part 3.

Part 3 — Deployment (localhost → public URL)

The target topology:



Three services, all on free tiers. The full runbook is in `DEPLOY.md`; the short version:

1. **Qdrant Cloud** — create a free cluster, copy its URL + API key.
2. **Render** — deploy the backend from `render.yaml` (Docker; Tesseract is baked into the image). Set `OPENROUTER_API_KEY`, `QDRANT_URL`, `QDRANT_API_KEY`, `CORS_ORIGINS`.

3. **Vercel** — deploy `frontend/` with **Root Directory = `frontend`**, Vite preset, and `VITE_API_URL` = the Render URL.
4. Add the Vercel URL to the backend's `CORS_ORIGINS` and redeploy.

Two properties of free-tier hosting to understand (and mention in an interview as *known trade-offs*, not surprises):

- **Cold starts.** Render's free web service sleeps after ~15 min idle; the first request wakes it (~50s). Acceptable for a demo.
- **Ephemeral disk.** The SQLite registry and uploaded files sit on the instance's disk, which resets on redeploy/sleep. Each visitor starts a fresh session. **Vectors persist in Qdrant Cloud regardless.** For durable history, move the registry to Postgres (noted as future work).

Part 4 — The bug journal (the interview gold)

Every bug below is real, in the order it mattered. Format: **Symptom** → **Root cause** → **Fix** → **Lesson.** The lessons generalize far beyond this project.

Bug #1 — OpenRouter embeddings crash on `null` data

Symptom. Embedding calls crashed. OpenRouter returned an embedding response whose `data` was `null` instead of a list of floats.

Root cause. The standard OpenAI-compatible embedding client defaults to `encoding_format="base64"`. OpenRouter doesn't return base64 the same way, so the decoded payload came back null → crash.

Fix. A thin custom embedding class (`openrouter_embedding.py`) that forces `encoding_format="float"`.

Lesson. "OpenAI-compatible" APIs are compatible *until they aren't.* When you integrate a drop-in replacement provider, the defaults that worked for the original may not hold. You only find these by actually calling the API and reading the raw response — not by trusting the SDK's defaults.

Bug #2 — Qdrant Cloud rejects filtered search: *"Index required but not found"*

Symptom. Every query worked perfectly on local Docker Qdrant, but on **Qdrant Cloud** every question failed. The backend returned HTTP 500 in ~1.9s (fast — so *not* a timeout). Qdrant's error was `400: Index required but not found for "claim_id"`.

Root cause. Retrieval filters results to the current claim with a metadata filter on `claim_id`. **Managed Qdrant Cloud refuses to filter on a field that has no payload index** — it returns 400. Local Docker Qdrant is lenient and filters an unindexed field anyway, so this bug was **invisible in development** and only appeared in production.

Fix. Create a `claim_id` **KEYWORD payload index** on the collection. Done in two places:

- immediately on the existing cloud collection (one-off), and
- durably in `ensure_collection()`, which now creates the index idempotently every startup so a fresh collection is always correct.

Lesson. *A managed service is not the same as the local dev version of the same software.* Managed tiers enforce constraints (indexes, quotas, auth) that your permissive local setup silently ignores. Test against something as close to prod as you can — and when a feature works locally but 400s in the cloud, suspect a constraint the cloud enforces that local doesn't.

Bug #3 — The 500 that looked like a network failure ("NetworkError")

Symptom. The browser showed a generic **"NetworkError"** with no detail — it *looked* like the backend was down or unreachable. It wasn't: the request reached the server and came back as a 500 (this was the surfacing of Bug #2).

Root cause. A raw, unhandled 500 in FastAPI **bypasses the CORS middleware**, so the error response carries **no `Access-Control-Allow-Origin` header**. The browser then refuses to expose the response to JavaScript and reports it as an opaque cross-origin "NetworkError" — hiding the real status and message.

Fix. Wrap the query handler so any unexpected exception becomes a proper `HTTPException` (a clean 500 with a readable `detail`). Because it's now a structured response, it flows **through** the CORS middleware and gets the CORS header — so the browser actually shows the real error. Specific upstream failures are mapped too: rate limit → 429, provider error → 502.

Lesson. In a cross-origin app, **an unhandled exception is worse than a handled error** — it strips CORS headers and turns a debuggable 500 into a mysterious "NetworkError." Always convert unexpected failures into structured responses. And when you see "NetworkError" with a request that clearly *did* reach the server, suspect a missing CORS header on an error path, not a dead server.

Design decisions that prevented bugs (worth mentioning too)

- **OCR only when needed.** Running OCR on every page would be slow and would *degrade* accuracy on clean digital text (OCR guesses; a text layer is exact). The per-page character-threshold fallback means digital pages use their real text and only true scans get OCR'd. Handling the mixed-content case (a PDF

with both digital and scanned pages) fell out of this design for free.
- **Per-page / per-row parsing.** Choosing the parse *unit* deliberately (page for PDF, row for Excel) is what makes citations precise and stops a benefits row from being chopped in half by the chunker.
- **Structural, not model-generated, citations.** Reading citations from metadata (not asking the LLM) means they can't be hallucinated.

Part 5 — Rapid-fire interview Q&A

Q: What is RAG and why use it? Retrieve relevant chunks from your own documents and feed only those to an LLM. Solves context-window limits, cost/latency, and hallucination in one move.

Q: Embedding model vs. chat model? Embedding model turns text into vectors to *find* relevant chunks; chat model *writes* the answer from them. Different models, different jobs.

Q: Why both a vector DB and SQLite? Qdrant does nearest-vector search; SQLite tracks sessions and processing status. Different access patterns — one store doing both makes both worse.

Q: How do you keep answers grounded / avoid hallucination? A system prompt that says "answer only from this context," plus retrieval that limits what the model sees, plus citations read structurally from chunk metadata.

Q: How are citations trustworthy? They're built from the metadata of the exact chunks that were retrieved (filename, page, similarity score) — not generated by the LLM, so they can't be faked.

Q: Why chunk with overlap? So a sentence's meaning isn't severed at a chunk boundary; the overlap preserves context across the cut.

Q: Why is upload asynchronous? Embedding a large PDF takes seconds to minutes; doing it inside the HTTP request would time out. Return `processing` immediately and let the client poll.

Q: A feature worked locally but broke in production — example? Qdrant metadata filtering. Local Docker filters an unindexed field; managed Qdrant Cloud requires a payload index and returns 400 without it (Bug #2).

Q: You saw "NetworkError" in the browser — how did you debug it? Confirmed the request reached the server (fast 500, not a timeout), realized an unhandled 500 skips CORS middleware and drops the CORS header, so the browser hid the real error. Converted unhandled exceptions to structured HTTP errors (Bug #3).

Q: Biggest lesson from deploying this? Managed/free-tier services enforce things local dev doesn't — indexes, auth on every client, quotas, cold starts, ephemeral disk. Most of my bugs were the gap between "works on my laptop" and "works in the cloud," not the RAG logic itself.

Companion docs: `ARCHITECTURE.md` *(how this codebase is wired) ·* `DEPLOY.md` *(the deploy runbook).*

Part 6 — Zero to RAG: a from-absolute-scratch course

Who this is for: you have *never written a line of code*, you don't know what an "API" or a "server" is, and you want to build RAG systems and configure LLMs. This part assumes **nothing**. It teaches the theory thoroughly and gives you **runnable code for every idea**. Read it top to bottom. Type the code yourself — don't copy-paste; typing is how it sticks.

> Everything above (Parts 1–5) is about *this specific project*. Part 6 is a standalone course. When you finish it, Parts 1–5 will read easily.

Table of contents for Part 6

- 6.0 — How to read this and what you'll be able to do
- 6.1 — The absolute basics: what is a program, Python, the terminal
- 6.2 — Theory: what an LLM actually is (tokens, weights, prediction, training)
- 6.3 — Theory: the context window, and why RAG has to exist
- 6.4 — Your machine: installing Python and making your first script run
- 6.5 — Talking to an LLM in code (your first API call)
- 6.6 — LLM configuration explained (temperature, top_p, max_tokens, and friends)
- 6.7 — Prompt engineering: system vs. user, and getting reliable output
- 6.8 — Theory: embeddings and vectors (the heart of retrieval)
- 6.9 — Code: turning text into vectors and measuring similarity by hand
- 6.10 — Theory: what a vector database is and why you need one
- 6.11 — **Build a tiny RAG from scratch** (no frameworks, ~70 lines, fully runnable)
- 6.12 — Graduating to a framework (what LlamaIndex does for you)
- 6.13 — Chunking, deeply (the single biggest lever on quality)
- 6.14 — How to tell if your RAG is any good (evaluation basics)
- 6.15 — A vocabulary list you can quiz yourself on
- 6.16 — A 4-week self-study plan and where to go next
- 6.17 — Watch it instead: free, top-tier video sources for every topic

---

6.0 — How to read this and what you'll be able to do

By the end of Part 6 you will:

- understand, in plain language, **what an LLM is and how it produces text**;
- be able to **call an LLM from a Python script** and control its behaviour with configuration;

- understand **embeddings and vector similarity** — the mechanism behind all retrieval and search;
- have **built a complete, working RAG system from scratch** with no framework, and understand every line;
- know what a **framework (LlamaIndex)** and a **vector database (Qdrant)** add, and when you actually need them.

Two rules for the whole part:

1. **Run everything.** Reading code teaches you 20%; running and breaking it teaches the other 80%.
2. **It's normal to not fully get something the first time.** Concepts here are circular — embeddings make more sense after you've seen retrieval, and vice versa. Keep going; it closes up.

6.1 — The absolute basics

What is a program? A text file containing instructions, written in a language a computer can follow, executed one line at a time from top to bottom.

What is Python? A programming language designed to be readable. It's the default language for AI work because almost every AI tool ships a Python interface. We use Python throughout.

What is the terminal (a.k.a. shell, command line)? A text window where you type commands to your computer instead of clicking. On macOS it's the "Terminal" app; on Windows, "PowerShell" or "Windows Terminal." You'll use it to run your programs. A command is just a word you type and press Enter on. For example, `python myscript.py` means "Python, run the file called myscript.py."

What is an API? "Application Programming Interface." Forget the jargon: an API is **a way for one program to ask another program (often over the internet) to do something and give an answer back.** When we "call the OpenRouter API," our little Python script sends a question over the internet to a big computer that runs the LLM, and that computer sends the answer back. You rent the model by the call instead of running it yourself.

What is JSON? A simple text format for structured data — the language APIs speak. It looks like this:

```
{ "name": "Bhavan", "age": 22, "skills": ["python", "rag"] }
```

Curly braces hold key–value pairs; square brackets hold lists. That's 90% of it. When an API answers, it usually answers in JSON, and your code picks out the parts it wants.

What is a "server" / "backend" / "frontend"?

- **Frontend** — what a user sees and clicks (the web page).
- **Backend / server** — a program running on some computer, waiting for requests and answering them (your Python code that does the RAG work).

- The frontend sends requests to the backend; the backend does the thinking and sends answers back. That's the whole relationship.

You don't need more than this to start.

6.2 — Theory: what an LLM actually is

This section has no code. Read it slowly; it's the foundation.

It predicts the next token

An **LLM (Large Language Model)** does exactly one thing: **given some text, it predicts what comes next.** That's it. Everything else — answering questions, writing code, translating — is that one trick applied cleverly.

- A **token** is a chunk of text, roughly ¾ of a word. "unbelievable" might be three tokens: `un`, `believ`, `able`. Models read and write in tokens, not letters or words. (Why it matters: pricing and length limits are counted in tokens, not words.)
- Given the tokens so far, the model outputs a **probability for every possible next token** — e.g. after "The capital of France is," the token "Paris" gets a very high probability. It picks one, appends it, and repeats. Text is generated **one token at a time**, each new token fed back in to predict the next. This looping is called **autoregressive** generation.

Where the "knowledge" comes from: weights and training

- A model is a giant mathematical function with billions of adjustable numbers called **weights** (also "parameters"). "26 billion parameters" means 26 billion such numbers.
- **Training** is the process of tuning those numbers by showing the model an enormous amount of text (much of the public internet, books, code) and, over and over, nudging the weights so its next-token predictions get better. This takes huge computers and is done once by the model's creators.
- **Inference** is when *you* use the finished model to generate text. When you "call an LLM," you're doing inference. Training = building the model (expensive, rare). Inference = using it (cheap, constant).
- Crucially, the knowledge is **frozen** into the weights at training time. The model has no memory of your last question and no live access to the internet or your files. It only knows (a) what was in its training data and (b) whatever text you put in front of it *right now*. **This last point is the entire reason RAG exists** — see 6.3.

Why it "hallucinates"

Because the model is a next-token predictor, it will always produce *fluent, plausible* text — even when it doesn't know the answer. It's not lying; it's doing its only job (predict a likely continuation) in the absence of real information. A confident wrong answer is called a **hallucination**. The fix is not "a smarter model" — it's **giving the model the real facts in the prompt** so the likely continuation is also the correct one. That's RAG.

6.3 — Theory: the context window, and why RAG must exist

Everything you send the model in one call — your question, plus any documents, plus instructions — is the **context** (or "prompt"). The model can only consider a limited amount of it at once. That limit is the **context window**, measured in tokens (e.g. 8,000 or 128,000 tokens).

Three consequences, which together *force* RAG into existence:

1. **You can't fit everything.** A 100-page policy is far larger than most context windows. You physically cannot paste it all in.
2. **Even when it fits, it's wasteful.** Sending 100 pages on every question is slow and costs money per token. You'd pay to re-read the whole book to answer one question.
3. **The model has no memory between calls.** Each API call starts fresh. It won't remember a document you sent an hour ago. Anything it should "know" must be in *this* prompt.

RAG (Retrieval-Augmented Generation) is the answer to all three: keep the documents in an external store, and for each question **retrieve only the few relevant passages** and put *those* in the context. You augment the model's generation with retrieved facts. Small window, low cost, no memory needed, and — because the real facts are right there in the prompt — far less hallucination.

That's the whole idea. The rest of Part 6 is *how* to build it.

6.4 — Your machine: install Python, run your first script

1. Install Python. Go to https://www.python.org/downloads/ and install Python 3.12+. On macOS you can instead use Homebrew: `brew install python@3.12`. Verify it in a terminal:

```
python3 --version       # should print Python 3.12.x
```

2. Make a folder for your experiments and enter it:

```
mkdir ~/rag-course && cd ~/rag-course
```

3. Create a virtual environment. A "venv" is a private sandbox for this project's installed libraries, so they don't collide with anything else on your computer. Make one and "activate" it:

```
python3 -m venv venv              # creates a folder called venv
source venv/bin/activate          # macOS/Linux — turns the sandbox on
# on Windows:  venv\Scripts\activate
```

When it's active you'll see `(venv)` at the start of your prompt. **You activate it once per terminal session** before working.

4. Write your first program. Create a file `hello.py` (any text editor — VS Code is the standard free one) with one line:

```
print("Hello — my first program.")
```

Run it:

```
python hello.py           # prints: Hello — my first program.
```

`print(...)` displays text. Congratulations — that's a program. Everything from here is just more instructions in the same file.

A 90-second Python primer (enough to read every code block in Part 6):

```
# A line starting with # is a comment — a note for humans, ignored by Python.

name = "Bhavan"            # a variable: a labelled box holding a value (text here)
count = 3                  # a number
skills = ["python", "rag"] # a list (ordered collection)
info = {"role": "dev"}     # a dict: key -> value pairs, like JSON

# A function is a reusable named block. It takes inputs and returns an output.
def greet(who):
    return "Hello, " + who

print(greet(name))         # Hello, Bhavan

# A loop repeats an action for each item in a list.
for skill in skills:
    print(skill)           # prints python, then rag

# An "if" makes a decision.
if count > 2:
    print("more than two")
```

That's genuinely enough to follow along. Learn the rest as you need it.

6.5 — Talking to an LLM in code (your first API call)

You won't run a model on your laptop — real ones need giant hardware. Instead you **rent** one through an API. We'll use **OpenRouter**, which gives you one account to reach many models, **including free ones** — the same choice this project made.

1. Get a key. Sign up at https://openrouter.ai, create an **API key** (a secret password-like string starting `sk-or-v1-...`). Treat it like a password: never paste it into code you'll share, never commit it to GitHub.

2. Store the key safely in an environment variable. An **environment variable** is a value your operating system holds outside your code, so secrets stay out of your files. Set it in your terminal:

```
export OPENROUTER_API_KEY="sk-or-v1-your-key-here"   # macOS/Linux
# Windows PowerShell:  $env:OPENROUTER_API_KEY="sk-or-v1-..."
```

3. Install the client library. A **library** is pre-written code you install and reuse. The `openai` library speaks the API format OpenRouter uses:

```
pip install openai
```

(`pip` is Python's installer; it downloaded and set up the library.)

4. Your first LLM call. Create `first_llm.py`:

```
import os                          # lets us read environment variables
from openai import OpenAI          # the client library

# Point the client at OpenRouter and hand it our secret key.
client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=os.environ["OPENROUTER_API_KEY"],   # read the secret we exported
)

# Ask the model something.
response = client.chat.completions.create(
    model="google/gemma-4-26b-a4b-it:free",     # which model to rent (a free one)
    messages=[
        {"role": "user", "content": "In two sentences, what is RAG?"}
    ],
)

# The API answers in a structured object; pull out the text.
print(response.choices[0].message.content)
```

Run it:

```
python first_llm.py
```

You just did **inference over the internet**: your script sent a prompt to a 26-billion-parameter model and printed its answer. Line by line:

- `messages` is the conversation. Each message has a **role** and **content**.
- `role: "user"` is you. (We'll meet `"system"` and `"assistant"` in 6.7.)
- `response.choices[0].message.content` is where the reply text lives — the API can return several choices; we take the first.

If it errors: a `401` means the key is wrong or not exported; a `429` means you've hit a rate limit (free tiers cap calls per day — wait and retry); no internet means no answer (the model lives on their computer, not yours).

6.6 — LLM configuration explained ("LLM configs")

When you call a model you can pass **settings that shape *how* it generates.** These are what people mean by "LLM configs." Here is each one, what it does, and when to change it.

```
response = client.chat.completions.create(
    model="google/gemma-4-26b-a4b-it:free",
    messages=[{"role": "user", "content": "Name three fruits."}],
    temperature=0.7,       # randomness / creativity     (0.0 – ~2.0)
    top_p=1.0,             # nucleus sampling             (0.0 – 1.0)
    max_tokens=256,        # cap on the length of the reply
    frequency_penalty=0.0, # discourage repeating the same tokens (-2 – 2)
    presence_penalty=0.0,  # encourage introducing new topics       (-2 – 2)
    stop=None,             # optional strings that force it to stop
    seed=None,             # (some providers) reproducibility
)
```

temperature — the one you'll actually tune

Recall the model outputs a *probability* for every next token. **Temperature** controls how boldly it picks.

- `temperature=0` → always take the single most likely token. Output is **deterministic and focused** — same input tends to give the same answer. Use this for **factual, extraction, or RAG** tasks where you want the safe, grounded answer. *For a RAG system like ClaimSage you want low temperature.*
- `temperature≈0.7` → sometimes pick a slightly-less-likely token. Output is **more varied and creative**, and less repetitive. Use for brainstorming, writing, ideation.
- `temperature≈1.5+` → wild, surprising, often incoherent. Rarely useful.

Mental model: temperature is a **creativity dial**. Low = careful and repeatable. High = imaginative and risky.

top_p — the other randomness knob (usually leave alone)

top_p ("nucleus sampling") says: only consider the most-likely tokens whose probabilities add up to `p`. `top_p=0.1` means "only the top 10% most-likely mass is even eligible." It's an *alternative* way to control randomness. **Rule of thumb: tune `temperature` *or* `top_p`, not both.** Most people leave `top_p=1.0` and just use temperature.

max_tokens — the length cap

The maximum number of tokens in the **reply**. The model stops when it hits this, even mid-sentence. Two reasons to set it: (1) you pay per token, so a cap controls cost; (2) it prevents runaway answers. Note it limits the *output*, separate from the context-window limit on the *input*.

frequency_penalty / presence_penalty — anti-repetition

- **frequency_penalty** > 0 makes tokens *less* likely the more they've already appeared — reduces the model looping "the the the" or repeating a phrase.
- **presence_penalty** > 0 nudges the model toward *new* topics it hasn't mentioned yet.
- You rarely need these. Reach for a small positive `frequency_penalty` only if you see repetitive output.

stop — hard stop strings

A list of strings that, if the model is about to produce one, cut the generation off immediately. Useful when you've told it to format output a certain way and want to end at a marker.

seed — reproducibility

Some providers let you pass a fixed **seed** so that (with `temperature=0`) you get the *same* output every run — handy for testing. Not all providers honour it.

The 30-second summary for a RAG builder: set `temperature=0` (grounded, repeatable), set a sensible `max_tokens`, leave the rest at defaults. That's a correct "LLM config" for document Q&A.

6.7 — Prompt engineering: system vs. user, reliable output

The `messages` list can hold three **roles**:

- **`system`** — instructions that set the model's behaviour and rules for the whole conversation. The user usually never sees it. This is where you put *"You are a careful assistant. Answer ONLY from the provided context. If the answer isn't there, say you don't know."*
- **`user`** — what the person asks.
- **`assistant`** — the model's own previous replies (you include past ones to give it memory of the conversation, since it has none on its own).

```
response = client.chat.completions.create(
    model="google/gemma-4-26b-a4b-it:free",
    temperature=0,
    messages=[
        {"role": "system", "content":
            "You are a precise assistant. Answer ONLY using the context provided "
            "by the user. If the answer is not in the context, reply exactly: "
            "'I don't know based on the documents.'"},
        {"role": "user", "content":
            "Context:\n'The knee surgery waiting period is 24 months.'\n\n"
            "Question: What is the waiting period for knee surgery?"},
    ],
)
print(response.choices[0].message.content)   # -> "24 months."
```

Notice what happened: we **put the fact in the prompt** and **told the model to use only that.** This tiny example is RAG in miniature — the "retrieval" was us pasting the relevant sentence. In a real system, code finds that sentence for you. That's the next few sections.

Practical prompt-engineering rules:

1. Put durable rules in the **system** message, the specific ask in **user**.
2. Be explicit about **what to do when the answer is missing** (prevents hallucination).
3. Ask for a specific **format** if you need to parse the output ("answer in one sentence," "return a bulleted list").
4. **Show, don't just tell** — a couple of examples in the prompt ("few-shot") dramatically improve reliability for tricky formats.
5. Test at **temperature 0** so you're debugging the prompt, not randomness.

6.8 — Theory: embeddings and vectors (the heart of retrieval)

Here's the question RAG has to answer efficiently: *"Out of thousands of document chunks, which few are relevant to this question?"* Keyword matching is too dumb — a question about "heart attack" should also find a chunk that says "myocardial infarction," which shares no words. We need to match by **meaning.** Embeddings are how.

A vector is a point in space

An **embedding** is a list of numbers — a **vector** — that represents a piece of text's *meaning*. For example (toy, 3 numbers):

```
"cat"    -> [0.9, 0.1, 0.2]
"kitten" -> [0.88, 0.12, 0.19]     # very close to "cat"
"finance"-> [0.05, 0.95, 0.4]      # far from both
```

Real embeddings have hundreds or thousands of numbers (this project's model produces **2048**). Each number is a **dimension**. You can't picture 2048-D space, but the rule is simple and it's the whole magic:

> **Texts with similar meaning get vectors that are close together; unrelated texts get vectors that are far apart.**

The embedding *model* (trained just like an LLM, but for this job) is what turns text into these meaning-coordinates. "heart attack" and "myocardial infarction" land near each other even with no shared words — because the model learned they *mean* the same thing.

Measuring "closeness": cosine similarity

To find relevant chunks we need to measure how close two vectors are. The standard measure is **cosine similarity**: it looks at the *angle* between two vectors, ignoring their length.

- Result ranges from **-1 to 1**.
- **1** = pointing the same direction = same meaning.
- **0** = unrelated.
- **-1** = opposite.

So retrieval becomes: embed the question, then find the stored chunk vectors with the **highest cosine similarity** to it. Those are your relevant passages. That's literally all "semantic search" is.

You don't have to take this on faith — you'll compute it by hand in the next section.

6.9 — Code: text → vectors, and similarity by hand

Install a tiny bit of math help and make the embedding call.

```
pip install numpy      # numpy = fast number/vector math for Python
```

Create `embeddings.py`:

```
import os
import numpy as np
from openai import OpenAI

client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=os.environ["OPENROUTER_API_KEY"],
)

def embed(text):
    """Turn a piece of text into its embedding vector (a list of numbers)."""
    resp = client.embeddings.create(
        model="nvidia/llama-nemotron-embed-vl-1b-v2:free",
        input=text,
        encoding_format="float",   # ask for plain numbers (see the note below!)
    )
    return np.array(resp.data[0].embedding)   # as a numpy array for easy math

def cosine_similarity(a, b):
    """How aligned are two vectors? 1 = identical meaning, 0 = unrelated."""
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

# Embed four texts.
cat     = embed("a small domestic cat")
kitten  = embed("a baby cat")
finance = embed("quarterly financial report")

print("cat vs kitten :", cosine_similarity(cat, kitten))    # high (~0.8+)
print("cat vs finance:", cosine_similarity(cat, finance))   # low  (~0.1–0.3)
```

Run it. You'll see **cat/kitten scores high and cat/finance scores low** — you just measured meaning with numbers. That single idea, scaled up, *is* how search, recommendations, and RAG retrieval work.

> **A real bug you'd hit here** — and one this project hit (Bug #1 in Part 4): that `encoding_format="float"` argument is not optional with some providers. The default (`"base64"`) can come back malformed from OpenRouter and crash. This is a perfect example of why you *run* the code: theory doesn't reveal this; execution does.

6.10 — Theory: what a vector database is (and when you need one)

In 6.9 you compared the question to a few vectors by looping over them. With **3 chunks** that's fine. With **50,000 chunks**, comparing the question to every single one on every query is too slow. A **vector database** (a.k.a. vector store) solves exactly this: it **stores millions of vectors and finds the nearest ones to a query almost instantly.**

- It uses clever indexing (approximate nearest-neighbour search) to avoid checking every vector — it gets *almost* the exact top matches, *much* faster.
- It stores, alongside each vector, some **metadata** (the original text, the filename, the page number). That's what lets you show *where* an answer came from — the citation.
- **Qdrant** (used in this project) is one such database. Others: Pinecone, Weaviate, Chroma, FAISS. They all do the same core job.

When do you actually need one? Not on day one. For a few hundred chunks, a Python list and the loop from 6.9 works. Reach for a real vector DB when you have lots of documents, want persistence (survive a restart), or need speed. In 6.11 you'll build RAG *without* one first — so you understand what the database is replacing.

6.11 — Build a tiny RAG from scratch (no framework)

Now assemble everything into a **complete, working RAG system** in one file — no LlamaIndex, no Qdrant, nothing hidden. If you understand this, you understand every RAG system on earth; the fancy ones just optimize these same steps.

Create `tiny_rag.py`:

```
import os
import numpy as np
from openai import OpenAI

client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=os.environ["OPENROUTER_API_KEY"],
)

CHAT_MODEL  = "google/gemma-4-26b-a4b-it:free"
EMBED_MODEL = "nvidia/llama-nemotron-embed-vl-1b-v2:free"

# ---------------------------------------------------------------------------
# Our "documents". In a real system these come from PDFs/Excel/etc. Here we
# just hard-code a few sentences so the whole pipeline fits on screen.
# ---------------------------------------------------------------------------
DOCUMENTS = [
    "The waiting period for knee surgery is 24 months from the policy start.",
    "Dental treatment is not covered under the basic plan.",
    "The annual coverage limit for hospitalization is $50,000.",
    "Maternity benefits apply after a continuous 9-month waiting period.",
    "Ambulance charges are reimbursed up to $200 per claim.",
]

def embed(text):
    resp = client.embeddings.create(
        model=EMBED_MODEL, input=text, encoding_format="float"
    )
    return np.array(resp.data[0].embedding)

def cosine(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

# ---- INGESTION: embed every document once and remember the vectors ----------
print("Embedding documents (ingestion)...")
STORE = [(doc, embed(doc)) for doc in DOCUMENTS]   # our tiny "vector database"

# ---- RETRIEVAL: find the top-k documents most similar to the question -------
def retrieve(question, k=2):
    q_vec = embed(question)
    scored = [(cosine(q_vec, vec), doc) for doc, vec in STORE]
    scored.sort(reverse=True)             # highest similarity first
    return scored[:k]                     # keep the best k

# ---- GENERATION: hand the retrieved chunks to the LLM, grounded -------------
def answer(question):
    top = retrieve(question, k=2)
    context = "\n".join(f"- {doc}" for score, doc in top)

    resp = client.chat.completions.create(
        model=CHAT_MODEL,
        temperature=0,                    # grounded, repeatable
        messages=[
            {"role": "system", "content":
                "Answer ONLY using the context. If it's not there, say "
```

```
                "'I don't know based on the documents.'"},
            {"role": "user", "content":
                f"Context:\n{context}\n\nQuestion: {question}"},
        ],
    )
    reply = resp.choices[0].message.content

    print(f"\nQ: {question}")
    print(f"A: {reply}")
    print("Sources used (with similarity):")
    for score, doc in top:
        print(f"   [{score:.2f}] {doc}")

# ---- Try it ----------------------------------------------------------------
answer("How long before knee surgery is covered?")
answer("Is dental included?")
answer("What is the capital of France?")   # not in the docs -> should decline
```

Run it:

```
python tiny_rag.py
```

Watch what happens:

- The knee question retrieves the knee sentence (high similarity) and the model answers "24 months" — **grounded in a real document**.
- The dental question finds the dental sentence and answers correctly.
- The France question retrieves *something* (the closest chunks) but none are relevant, so — because of the system prompt — the model says it doesn't know **instead of hallucinating.** That refusal is the system working correctly.

You have now built a RAG system. Map it back to the two-phase picture from 6.2 / Part 1:

Concept	Line in `tiny_rag.py`
Ingestion (embed + store)	`STORE = [(doc, embed(doc)) ...]`
The vector "database"	the `STORE` list
Retrieval (nearest by cosine)	`retrieve()`
Grounding prompt	the `system` message
Generation	the `chat.completions.create` call
Citations	printing `score` + `doc`

The real ClaimSage is this exact pipeline with production parts swapped in: PDFs instead of hard-coded strings, Qdrant instead of a Python list, a web API and UI on top, and OCR for scans. **Same skeleton.**

6.12 — Graduating to a framework (what LlamaIndex does)

Your `tiny_rag.py` works, but it's missing things real documents need: reading PDFs/Excel/email, splitting long text into chunks, handling thousands of vectors, retries, metadata. Writing all that yourself is a lot of plumbing.

A **framework** is pre-built plumbing. **LlamaIndex** (used in this project) gives you:

- **Readers** that turn PDFs, Word, Excel, email into text for you.
- **Node parsers** that chunk text sensibly (with overlap — see 6.13).
- **Vector-store integrations** so "store these" and "find similar" become one line, whether the store is Qdrant, Pinecone, or in-memory.
- A **query engine** that wires retrieval → prompt → LLM → response with citations.

Roughly, your 70-line file becomes:

```
# Conceptual — the framework hides the loop you wrote by hand.
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

documents = SimpleDirectoryReader("./my_pdfs").load_data()  # read + parse files
index = VectorStoreIndex.from_documents(documents)          # chunk + embed + store
engine = index.as_query_engine()                            # retrieval + LLM wired
print(engine.query("How long before knee surgery is covered?"))
```

The framework isn't doing anything *conceptually* new — it's doing precisely what you built in 6.11, just robustly and for real file types. **This is why you built the tiny version first:** now the framework is a convenience, not a mystery. When it misbehaves (and it will), you know what it's *supposed* to be doing underneath.

That's also the honest trade-off: frameworks save time but hide detail. Because you did 6.11, you can debug them. Someone who skipped it can't.

6.13 — Chunking, deeply (your biggest quality lever)

A whole 20-page document is too big to be one vector — its meaning gets averaged into mush, and you can't cite a page. So before embedding, you **chunk**: split text into bounded pieces. How you chunk is the **single biggest influence on RAG quality**, more than the model choice. Get this right and mediocre models shine; get it wrong and great models fail.

Key parameters:

- **Chunk size** (e.g. ~512 tokens). Too big → each chunk mixes several topics, so retrieval is imprecise and you waste context. Too small → a single idea gets split across chunks and no one chunk contains the full answer. ~200–500 tokens is a common sweet spot for prose.

- **Chunk overlap** (e.g. ~50 tokens). Consecutive chunks share a little text at the seam. Why: a sentence answering the question might sit exactly on a boundary; overlap ensures it appears *whole* in at least one chunk instead of being sliced.

Smarter chunking = respect the document's structure. Don't blindly cut every N characters. This project chooses the **unit of splitting deliberately**:

- **PDF → one chunk per page**, so a citation can name the page.
- **Excel → one chunk per row**, so a "benefits schedule" row is never chopped in half (which would destroy its meaning).
- Long prose → size + overlap as above.

A quick mental test for a chunk: *"If retrieval returned only this piece, could the model answer a reasonable question from it?"* If a chunk is half a sentence or three unrelated topics, fix your chunking.

When answers come back vague or wrong, **suspect chunking before you blame the model.** It's the most common real cause.

6.14 — How to know if your RAG is any good (evaluation)

"It looked right when I tried it" is not evaluation. To improve a RAG system you need to *measure* it. The beginner-friendly method:

1. **Write a test set.** 15–30 questions about your documents, each with the correct answer *and* the page/chunk it should come from. This is your "answer key."
2. **Measure retrieval separately from generation.** Two different things can go wrong:
    - **Retrieval quality** — did the right chunk make it into the top-k at all? If the correct chunk wasn't retrieved, the LLM never had a chance. (Metrics: *recall* = fraction of questions whose correct chunk was retrieved.)
    - **Answer quality** — given the right chunks, was the final answer correct and grounded? (You can judge by hand, or have a strong LLM grade answers against your key — "LLM-as-judge.")
3. **Change one thing at a time** — chunk size, top-k, the prompt, the model — and re-run the same test set. Keep whatever moved the numbers up. This is the whole game: measure, change one variable, re-measure.

Why split retrieval from generation? Because the fixes are opposite. Bad retrieval → fix chunking/embeddings/top-k. Good retrieval but bad answers → fix the prompt or the model. Measuring them together hides which one is broken. (This project lists an "eval harness" as planned phase-2 work for exactly this reason.)

6.15 — Vocabulary you can quiz yourself on

- **LLM** — a model that predicts the next token; used to generate text.
- **Token** — a chunk of text (~¾ word); the unit models read/write and you're billed in.
- **Inference** — using a trained model to generate output (what an API call does).
- **Training** — the expensive one-time process of tuning a model's weights.
- **Weights / parameters** — the billions of numbers that *are* the model's knowledge.
- **Hallucination** — a confident, fluent, wrong answer; RAG's main target.
- **Context window** — the max tokens a model can consider at once.
- **Prompt / context** — everything you send in one call (system + user + docs).
- **System / user / assistant** — the three message roles; system sets the rules.
- **Temperature** — randomness dial; 0 = focused/repeatable, high = creative.
- **top_p, max_tokens, frequency/presence penalty** — the other generation configs.
- **Embedding** — a vector representing text's meaning.
- **Vector / dimension** — the list of numbers, and how many there are.
- **Cosine similarity** — angle-based closeness of two vectors (-1..1); how we rank relevance.
- **Semantic search** — finding text by meaning via embedding similarity.
- **Chunk / node** — a bounded piece of a document that gets embedded and stored.
- **Chunk overlap** — shared text between adjacent chunks so ideas aren't split.
- **Vector database / store** — stores many vectors and finds nearest ones fast (Qdrant).
- **Metadata** — extra info stored with a vector (filename, page) → enables citations.
- **Retrieval** — fetching the most relevant chunks for a query.
- **Grounding** — forcing the answer to come from retrieved context.
- **RAG** — retrieve relevant chunks, then generate an answer grounded in them.
- **API** — a way for your code to ask another program (the model) to do work.
- **Environment variable** — a secret/config value kept outside your code.
- **venv** — a per-project sandbox for installed libraries.

6.16 — A 4-week self-study plan and where to go next

You don't learn this by reading — you learn it by building. A realistic plan:

Week 1 — Comfort with the tools. Install Python, make the venv, run `hello.py`. Do the 6.1 Python primer until variables/lists/functions/loops feel natural. Make your first LLM call (6.5). Change the question, change the model.

Week 2 — LLM control + embeddings. Work through every config in 6.6 — actually *see* what `temperature=0` vs `1.5` does to the same prompt. Do the embeddings script (6.9); embed your own sentences and predict the similarity scores before running.

Week 3 — Build RAG from scratch. Type out `tiny_rag.py` (6.11) yourself. Then *extend* it: add your own documents; make it read a real `.txt` file from disk and split it into chunks (6.13); raise `k` and watch answers change. Break it on purpose and fix it.

Week 4 — Go real. Rebuild the same thing with **LlamaIndex** (6.12) reading an actual PDF. Then read Parts 1–5 of this guide — you'll now recognize every piece — and read this repo's `ARCHITECTURE.md`. Optionally deploy something small.

After that: pick ONE real problem you personally care about (your notes, a textbook, a set of PDFs) and build a RAG over it end to end. A finished small project teaches more than ten tutorials.

Guiding principles to carry with you:

- **Run everything; break things on purpose.** Understanding comes from failure you diagnosed, not code you read.
- **Build the simple version by hand before reaching for a framework.** You can't debug a black box you never opened.
- **Change one variable at a time and measure.** That's how you actually improve a system instead of guessing.
- **When answers are bad, suspect (in order): chunking, retrieval, then the prompt — and only last, the model.**

Now go back to the top of this document and read Parts 1–5. What was jargon an hour ago is now your working vocabulary.

6.17 — Watch it instead: free, top-tier video sources for every topic

Some people learn faster by watching. Below, **every topic in Part 6 is mapped to several free, high-quality videos/courses** so you can pick the style that suits you. All of these are **genuinely free** (YouTube, or free-to-audit courses).

How to use this list: creators rename videos and links rot, so each entry is given as **"*Title*" — Creator**, plus a stable channel/site link. If a link ever breaks, search YouTube for the exact title + creator and you'll find it. Pick **one** option per topic — they overlap on purpose so you can choose a teaching style, not watch all of them.

> Legend: 🎬 single video · 📺 playlist/series · 🎓 full free course.

Foundations — terminal, Python setup, basic Python (§6.1, §6.4)

- 🎓 **"CS50's Introduction to Programming with Python" (CS50P)** — Harvard / David Malan. The gold-standard free beginner course. https://cs50.harvard.edu/python (also full on YouTube: https://youtube.com/@cs50)
- 🎬 **"Python for Beginners – Full Course"** — freeCodeCamp. https://youtube.com/@freecodecamp
- 🎬 **"Python Tutorial for Beginners (Full Course)"** — Programming with Mosh. https://youtube.com/@programmingwithmosh
- 🎞️ **"Python Tutorial for Beginners" (playlist)** — Corey Schafer. Clear, bite- sized. https://youtube.com/@coreyms
- 🎬 For the terminal specifically: **"Command Line Crash Course"** — freeCodeCamp (or search "Terminal / shell tutorial for beginners").

What an LLM actually is — how they work, theory (§6.2)

- 🎬 **"Intro to Large Language Models" (1 hr)** — Andrej Karpathy. The single best plain-English overview; start here. https://youtube.com/@AndrejKarpathy
- 🎬 **"Deep Dive into LLMs like ChatGPT" (~3.5 hr)** — Andrej Karpathy. Longer, more complete, still beginner-approachable.
- 🎞️ **"Neural Networks" series** incl. **"But what is a GPT? / Transformers, explained visually"** — 3Blue1Brown. Best visual intuition on the planet. https://youtube.com/@3blue1brown
- 🎬 **"Transformers, explained clearly"** & the **Neural Networks** series — StatQuest with Josh Starmer. Gentle, step-by-step. https://youtube.com/@statquest
- 🎬 Various short **LLM / GPT explainers** — Computerphile. https://youtube.com/@Computerphile

Tokens & the context window (§6.2, §6.3)

- 🎬 **"Let's build the GPT Tokenizer"** — Andrej Karpathy. Exactly what a token is, from scratch. (Channel above.)
- Covered well inside Karpathy's **"Intro to Large Language Models"** and the 3Blue1Brown transformer series above — if you watched those, you already have it.

Calling an LLM from code / what an API is (§6.5, and §6.1's "what's an API")

- 🎬 **"APIs for Beginners – How to use an API (Full Course)"** — freeCodeCamp. https://youtube.com/@freecodecamp
- 🎬 **"OpenAI API Python Tutorial"** — Tech With Tim (or search freeCodeCamp "OpenAI API course"). https://youtube.com/@TechWithTim
- 📄 Free official docs that double as tutorials: **OpenRouter Docs** (https://openrouter.ai/docs) and **Anthropic / OpenAI API quickstarts**.

LLM configuration — temperature, top_p, sampling (§6.6)

- 🎬 **"How to sample from language models" / "Temperature, top-k, top-p explained"** — search this title on YouTube; several excellent short explainers exist. Good channels: StatQuest, Computerphile.

- 📄 **OpenAI / Anthropic API reference on sampling parameters** — the authoritative, free description of each knob (temperature, top_p, penalties, max_tokens).
- Karpathy's **"Let's build GPT from scratch"** shows sampling/temperature in real code near the end.

Prompt engineering (§6.7)

- 🎓 **"ChatGPT Prompt Engineering for Developers"** — DeepLearning.AI (Andrew Ng & Isa Fulford). ~1 hr, free, hands-on, the best starting point. https://deeplearning.ai/short-courses
- 📄 **Anthropic's Prompt Engineering guide / interactive tutorial** — free, superb, practical. (In the Anthropic docs.)
- 🎬 **"Prompt Engineering Tutorial – Full Course"** — freeCodeCamp.

Embeddings, vectors & cosine similarity (§6.8, §6.9)

- 🎬 **"Word Embedding and Word2Vec, Clearly Explained"** — StatQuest. The friendliest intro to "text → numbers." https://youtube.com/@statquest
- 🎬 **"Vectoring Words (Word Embeddings)"** — Computerphile. Short and intuitive.
- 🧱 The **embeddings/attention portion of the Transformer series** — 3Blue1Brown. Best geometric intuition for "meaning as direction in space."
- 🎬 Search **"What are embeddings" — freeCodeCamp / Assembly AI** for a practical, code-first take.

Vector databases (§6.10)

- 🎬 **"Vector databases explained"** — search this title; **Qdrant**, **Pinecone**, and **Weaviate** each publish free, high-quality explainer videos and docs (Qdrant: https://youtube.com/@qdrant · docs https://qdrant.tech/documentation).
- 🎬 **"Vector Database Crash Course"** — freeCodeCamp.

RAG end-to-end — the whole pipeline (§6.11, §6.12)

- 🎓 **"Building and Evaluating Advanced RAG Applications"** — DeepLearning.AI (with LlamaIndex & TruEra). Free, and directly relevant to this project. https://deeplearning.ai/short-courses
- 🎬 **"Learn RAG From Scratch – Full Course"** — freeCodeCamp (built with the LangChain team). A long, complete, free RAG walkthrough. https://youtube.com/@freecodecamp
- 🎬 **"RAG explained"** short overviews — IBM Technology (https://youtube.com/@IBMTechnology) is excellent for crisp 5–10 min concept videos.

LlamaIndex specifically (§6.12)

- 📄🧱 **LlamaIndex official docs + YouTube** — starter tutorials that mirror §6.12. https://docs.llamaindex.ai · https://youtube.com/@LlamaIndex
- 🎓 The DeepLearning.AI **"Building and Evaluating Advanced RAG"** course above uses LlamaIndex throughout.

Chunking / text splitting (§6.13)

- 🎬 **"The 5 Levels of Text Splitting for Retrieval"** — Greg Kamradt. The canonical video on chunking; watch this one. (Search the title on YouTube.)
- Chunking is also covered hands-on inside the DeepLearning.AI RAG course and the freeCodeCamp RAG course above.

Evaluating a RAG system (§6.14)

- 🎓 **"Building and Evaluating Advanced RAG Applications"** — DeepLearning.AI. The evaluation half of this course is exactly §6.14 in practice.
- 🎓 **"Automated Testing for LLMOps"** & other free **DeepLearning.AI short courses** — good next steps once you want to measure quality seriously.

One-stop mega-resources (cover almost everything above)

- 📓 **"Neural Networks: Zero to Hero"** — Andrej Karpathy. If you want to *truly* understand what's under the hood, this free playlist builds a language model from nothing. Deeper than you need for RAG, but unmatched.
- 🎓 **DeepLearning.AI Short Courses** — a whole library of free, ~1 hr, hands-on courses on LLMs, RAG, embeddings, agents. https://deeplearning.ai/short-courses
- 🎓 **freeCodeCamp** (https://youtube.com/@freecodecamp) — full-length free courses on Python, APIs, embeddings, vector DBs, and RAG.

Suggested minimal watch-path if you only pick a few: CS50P (Python) → Karpathy "Intro to Large Language Models" → StatQuest "Word Embeddings" → DeepLearning.AI "ChatGPT Prompt Engineering for Developers" → freeCodeCamp "Learn RAG From Scratch." That sequence alone takes you from zero to building RAG.

End of Part 6. Companion docs: `ARCHITECTURE.md` · `DEPLOY.md`.